



BSc (Hons) in **Computer Science**
SE3062 : Intelligent Systems

Year 3 · Semester 1 · 2026 Semester 2

Faculty of Computing

Practical 02

Objective & Lecture Mapping: In Lecture 03, we learned that a mathematically perfect "Table-Driven Agent" is impossible to build due to combinatorial explosion. Instead, we must write Agent Programs. Today, you will implement a **Simple Reflex Agent** using Condition-Action rules, observe its fatal flaws in a partially observable environment, and then upgrade it to a **Model-Based Agent** that maintains an internal memory state.

Part 1: Practical Implementation — Coding the Architectures

Step 1.1: Setting the Trap (Partial Observability)

To see why Simple Reflex agents fail, we must handicap your agent's sensors. We are moving from a Fully Observable world to a Partially Observable one.

1. Open your `visual_grid_game.py` from Lab 01.
2. Locate the `get_percept(self)` method.
3. Modify it so it **no longer returns** the exact global coordinates (`agent_pos`). Instead, it should only return local booleans, e.g., `{'wall_ahead': True/False, 'food_here': True/False}` based on checking the adjacent cell in its current facing direction.

Step 1.2: The Simple Reflex Agent (Implementation & Failure)

1. Create a new class called `SimpleReflexAgent` .
2. Implement a `sense_and_act(self, percept)` method that uses strictly IF-THEN logic (Condition-Action rules). *Do not use an `__init__` method to store history.*

Example Logic: IF food_here THEN suck; IF wall_ahead THEN turn_left; ELSE move_forward

3. Run the simulation. **Observe:** Watch the agent get trapped in a corner or a U-shaped wall, infinitely repeating a cycle (e.g., turn left, step forward, hit wall, turn left...).

Step 1.3: The Model-Based Agent (Memory & State)

1. Create a new class called `ModelBasedAgent`.
2. Implement an `__init__(self)` method to initialize an internal memory state (e.g., `self.visited_cells = set()` or a relative movement tracker).
3. In `sense_and_act(self, percept)`, first **update the state** (Transition & Sensor Model) by recording the current percept and the last action taken.
4. Update your IF-THEN rules to query the memory.
Example Logic: IF wall_ahead AND left_is_visited THEN turn_right
5. Run the simulation. **Observe:** The agent should now remember where it has been, realize it is in a loop, and choose an alternate path to escape.

Part 2: Theoretical Evaluation & Lecture Mapping

Based on your practical observations above, answer the following questions to map your code directly to the architectural theories covered in Lecture 02.

1. (Remember) According to Lecture 02, why is it impossible to program a mathematically perfect "Table-Driven Agent" for complex environments like Chess? What happens as the agent's lifetime increases?

A Table-Driven Agent stores a lookup table mapping every historical percept sequence to an action. For an agent with a set of percepts $|P|$ operating over lifetime steps (T), the required table size is $(|P|^T)$.

As the agent's lifetime (T) increases, the table suffers from Combinatorial Explosion, growing exponentially. In Chess, with over 10^{40} board states and 10^{120} possible game sequences (The Shannon Number), storing such a lookup table is physically impossible because it exceeds the total storage capacity of the universe. Therefore, AI systems must use Agent Architectures that calculate actions dynamically on the fly rather than looking them up in a pre-computed table.

2. (Understand) Look at the code you wrote for your `SimpleReflexAgent` . Identify and explain the specific lines of code that represent the "Condition-Action Rules" discussed in the lecture.

In agent.py file, lines 25–30 represent the Condition-Action Rules:

Line 25–26 (if percept.get('food_here', False): return 'Up'): Maps the immediate condition food present directly to the action advance/collect.

Line 27–28 (elif percept.get('wall_ahead', False): return 'Left'): Maps the immediate condition obstacle ahead directly to the action turn left.

Line 29–30 (else: return 'Up'): Provides the default action move forward when no specific condition is triggered.

These lines implement pure reflex behavior because they trigger actions based strictly on the immediate percept dictionary input with zero memory or historical tracking.

3. (Analyze) Your `SimpleReflexAgent` likely got stuck in an infinite loop during Step 1.2. Based on the lecture, analyze exactly why this happened. How did the combination of "Partial Observability" and a lack of "Percept History" cause this failure?

The fatal flaw of a Simple Reflex Agent is that it requires a Fully Observable environment to function correctly. Because a reflex agent has zero memory, when critical state data is hidden (Partially Observable), different world states generate identical sensor readings. Without a percept history to track past actions, the agent's deterministic IF-THEN rules execute the exact same response repeatedly, trapping the agent in unavoidable infinite loops.

4. (Evaluate) In Step 1.3, you added an internal state to your `ModelBasedAgent` . Evaluate how your specific code handles the "Transition Model" (how the world evolves) and the "Sensor Model" (how the agent's actions affect the world).

In `ModelBasedAgent`:

Sensor Model (Lines 48–56): Predicts how actions change position by modifying `self.current_pos` based on `self.last_action`.

Transition Model (Lines 58–60): Updates world state by storing updated coordinates in `self.visited_cells` and appending percepts to `self.percept_history`.

Pre-Decision Update: The agent updates its position and memory at the beginning of `sense_and_act()` before rule matching. When a wall is encountered, it queries `self.last_action` to pick a different direction (e.g., choosing 'Right' if 'Left' was last taken), preventing infinite loops.

Finalize and Lock Lab 02 Documentation

Incomplete: 0/11 questions answered. Please provide detailed responses for all questions.

